

Lecture 4

Array Sorting, Merging



PREPARED BY:

**AFSANA BEGUM
SENIOR LECTURER,
SOFTWARE ENGINEERING DEPARTMENT,
DAFFODIL INTERNATIONAL UNIVERSITY.**

Contents



Bubble Sort

Sorting



There are different kinds of Sorting. Today we will see a new kind of sorting called “Bubble Sort”.

Some other kinds of sorting are:

- a. Quick Sorting
- b. Insertion Sorting
- c. Selection Sorting
- d. Merge Sorting
- e. Radix Sorting.

Bubble Sort

Bubble Sorting is an algorithm in which we are comparing first two values and put the larger one at higher index.

It takes next two values compare these values and place larger value at higher index.

This process do iteratively until the largest value is not reached at last index. Then start again from zero index up to $n-1$ index.

The algorithm follows the same steps iteratively until elements are not sorted.

Bubble sort is also known as exchange sort. Bubble sort is a simplest sorting algorithm

Sorting : Bubble sort



Sorting takes an unordered collection and makes it an ordered one.

1	2	3	4	5	6
77	42	35	12	101	5



1	2	3	4	5	6
5	12	35	42	77	101

→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**

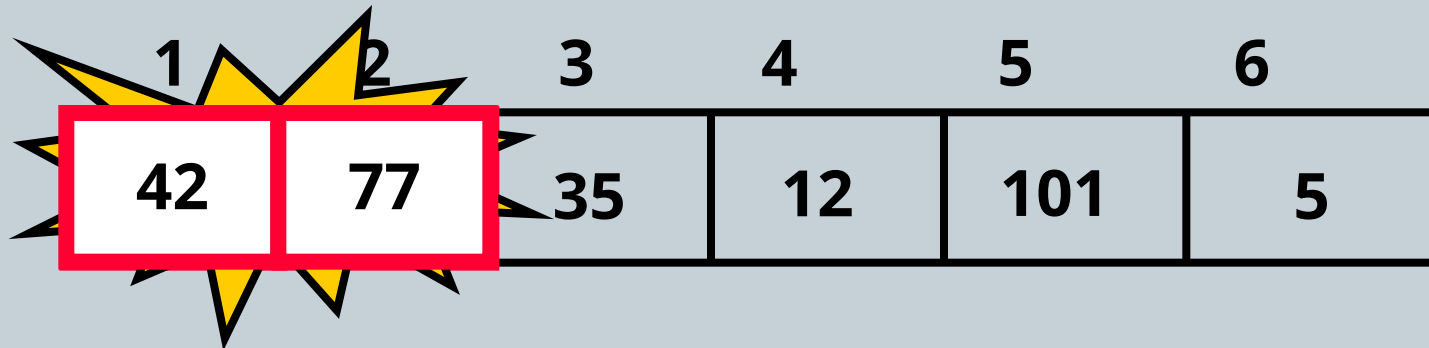
1	2	3	4	5	6
77	42	35	12	101	5

→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping

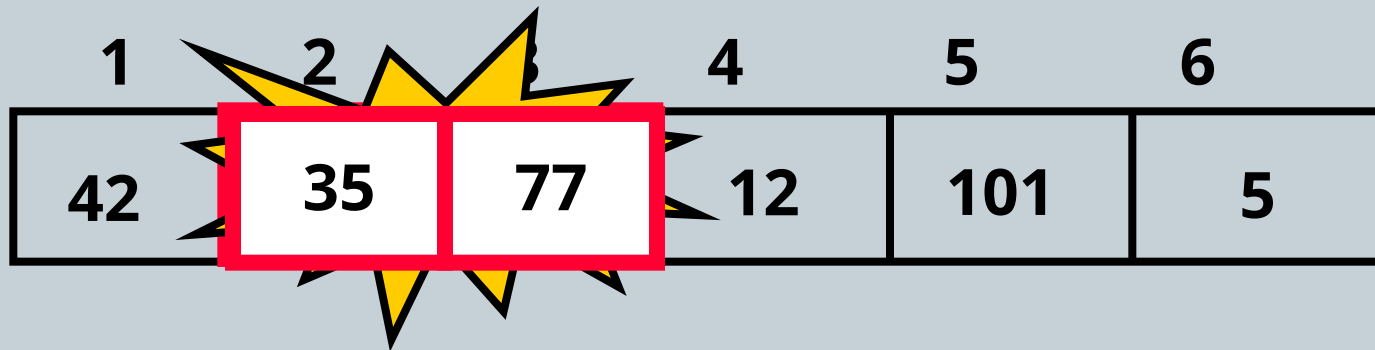


→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping

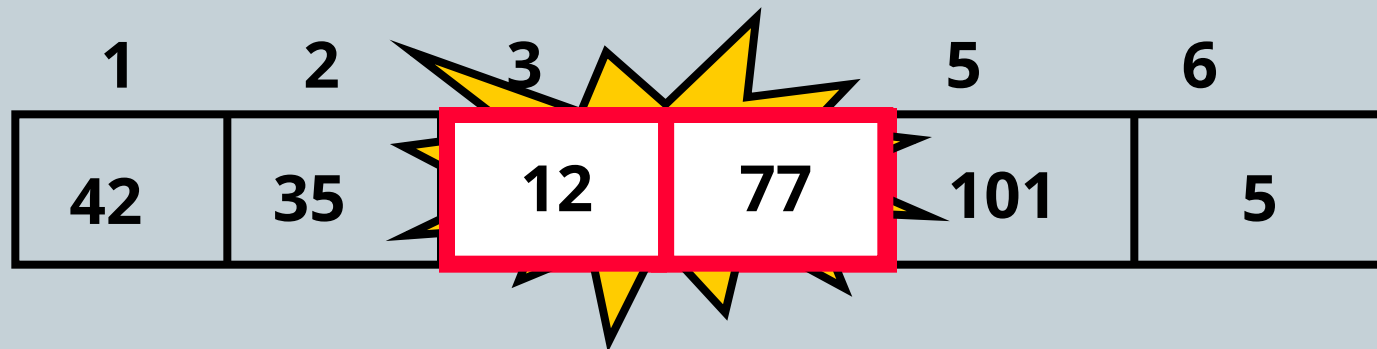


→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping

1	2	3	4	5	6
42	35	12	77	101	5

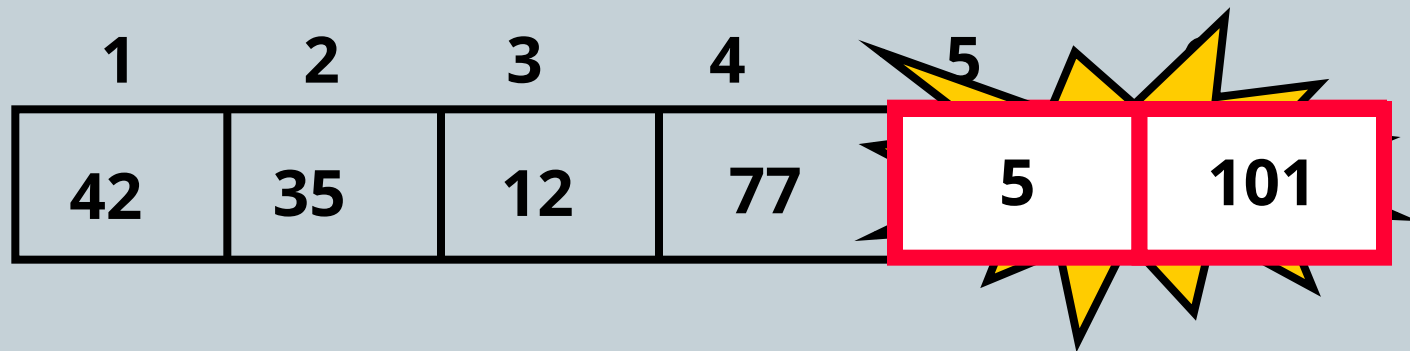
No need to swap

→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping



→ Bubbling Up → the Largest Element



Traverse a collection of elements

- Move from the front to the end
- “Bubble” the largest value to the end using pair-wise comparisons and swapping

1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

→ Bubbling Up → the Largest Element



Notice that only the largest value is correctly placed

All other values are still out of order

So we need to repeat this process

1	2	3	4	5	6
42	35	12	77	5	101

After n-1 comparisons

Largest value correctly placed

Repeat “Bubble Up” How Many Times?



If we have N elements...

And if each time we bubble an element, we place it in its correct location...

Then we repeat the “bubble up” process $N - 1$ times.

This guarantees we'll correctly place all N elements.

↖
|
↘

Reducing the Number of Comparisons



1	2	3	4	5	6
77	42	35	12	101	5

1	2	3	4	5	6
42	35	12	77	5	101

1	2	3	4	5	6
35	12	42	5	77	101

1	2	3	4	5	6
12	35	5	42	77	101

1	2	3	4	5	6
12	5	35	42	77	101

Main Part of Codes



Let int array[10]={...}, c,d; Here n=10 , Then :

```
for (c = 0 ; c < ( n - 1 ); c++)
{
    for (d = 0 ; d < n - c - 1; d++)
    {
        if (array[d] > array[d+1])

        /* For decreasing order use < */

        {
            swap = array[d];
            array[d] = array[d+1];
            array[d+1] = swap;
        }
    }
}
```

Summary



“Bubble Up” algorithm will move largest value to its correct location (to the right)

Repeat “Bubble Up” until all elements are correctly placed:

- **Maximum of $N-1$ times**

We reduce the number of elements we compare each time one is correctly placed



Questions?